Experiment No. : 6

Title: Authenticate and Consume REST APIs Using API Keys, Parameters, Pagination and API Documentation

Batch: B3 **Roll No.: 16014225801** **Experiment No.: 5**

Aim
To authenticate and consume REST APIs using API keys, query parameters, pagination, and API documentation, and to process the JSON responses returned by the API.

Resources Required
- Python 3.x
- Selenium
- VS Code / PyCharm / Jupyter Notebook
- Web browser – Chrome/Edge/Firefox

Target Website: Mention here the URL of your selected target website. Selected website should allow good amount of data to scrap and explore

Theory:

Theory
1. REST API
A **REST API** allows applications to communicate with a web service through HTTP requests.
A typical API request contains:
Client → HTTP Request → API Server
Client ← HTTP Response ← API Server
For example:
GET https://api.example.com/data
The server processes the request and returns a response, commonly in **JSON** format.

---

2. API Documentation
Before consuming an API, the developer should study its documentation.

Important information to identify:

Item	Purpose
Base URL	Root address of the API
Endpoint	Specific resource to access
HTTP Method	GET, POST, PUT, DELETE
Authentication	API key, Bearer token, OAuth etc.
Parameters	Values used to customize the request
Response format	JSON/XML
Pagination	Method for retrieving multiple pages
Rate limits	Restrictions on number of requests

Example:
Base URL:
https://api.example.com

Endpoint:
/weather

Method:
GET

Authentication:
API Key

Parameters:
city, units

Response:
JSON

---

3. API Key Authentication

An **API key** is a credential provided by an API service to identify and authorize a client.
A key may be supplied through:

Query parameter

```
params = {
   "city": "Mumbai",
   "apikey": API_KEY
}
```

HTTP header

```
headers = {
   "X-API-Key": API_KEY
}
```

The exact method depends on the API documentation.

Important: Students should never publish a real API key in a public notebook, GitHub repository, screenshot, or report.

---

4. Query Parameters

Query parameters allow the client to customize an API request.
For example:
https://api.example.com/weather?city=Mumbai&units=metric
Here:
city = Mumbai
units = metric
Using requests:

```
params = {
   "city": "Mumbai",
   "units": "metric"
}

response = requests.get(API_URL, params=params)
```

The requests library automatically constructs the query string.

5. JSON Response

Many REST APIs return data in JSON format.

Example:

```
{
   "city": "Mumbai",
   "temperature": 29,
   "humidity": 78
}
```

Python can convert the JSON response into a dictionary using:

```
data = response.json()
```

Individual values can then be accessed:

```
print(data["city"])
print(data["temperature"])
print(data["humidity"])
```

K. J. SOMAIYA COLLEGE OF ENGG.

6. Pagination

APIs may not return all records in a single request.

For example:

Page 1 → Records 1–10
Page 2 → Records 11–20
Page 3 → Records 21–30

A client can request successive pages:

```
for page in range(1, 4):

   params = {
      "page": page
   }

   response = requests.get(API_URL, params=params)

   data = response.json()

   print("Page:", page)
   print(data)
```

The exact pagination parameters depend on the API documentation.

Procedure

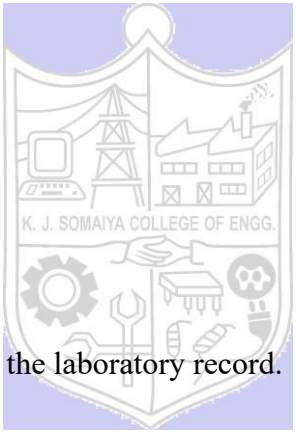
Step 1

Select an API that supports:

- REST requests
- API-key authentication
- query parameters
- JSON responses
- pagination

Step 2
Read the API documentation carefully.

Identify:
- Base URL
- Endpoint
- HTTP method
- Authentication mechanism
- Required parameters
- Optional parameters
- Response structure
- Pagination mechanism



Step 3
Store the API key securely.

For demonstration:
API_KEY = "YOUR_API_KEY"
Do **not** use or publish an actual key in the laboratory record.

Step 4
Import the requests library.
```
import requests
```

Step 5
Define the API endpoint.
```
API_URL = "YOUR_API_ENDPOINT"
```

Step 6
Prepare the authentication information according to the API documentation.

For example:
```
params = {
   "apikey": API_KEY
}
```

Step 7
Add query parameters.

Step 8
Send the GET request.

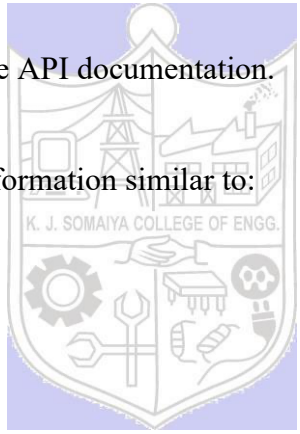
Step 9
Check the HTTP status code.

Step 10
Convert the response into JSON.

Step 11
Extract relevant information from the response.

For example:
print(data["city"])
print(data["temperature"])
The exact keys depend on the selected API.

Step 12
Implement pagination according to the API documentation.



Expected Output
A typical execution should display information similar to:
Page: 1
Status Code: 200

Page: 2
Status Code: 200

Page: 3
Status Code: 200

Total records collected: 30
The actual output will depend on the selected API and its response structure.

Observation Table

Sr. No.	Parameter	Observation
1	API Endpoint	Identified from API documentation
2	HTTP Method	GET
3	Authentication	API Key
4	Response Format	JSON
5	Query Parameters	As specified by API
6	Status Code	200 for successful request
7	Pagination	Multiple pages retrieved
8	Records Retrieved	Depends on API

Error Handling
Students should check the response before processing it.

```
if response.status_code == 200:
    data = response.json()
    print(data)
else:
    print("Error:", response.status_code)
```

```
print(response.text)
```

Common status codes:

Status Code	Meaning
200	Successful request
400	Bad request
401	Authentication failure
403	Access forbidden
404	Resource not found
429	Too many requests
500	Server-side error

AI-Based Post-Lab Questions

AI Use Case	Post-Lab Question
1. Climate Intelligence for Heatwave Monitoring, Prediction, and Early Warning	**Q1.** A climate-monitoring system collects temperature, humidity, and other weather observations from an authenticated weather API. Explain how **API-key authentication, query parameters, JSON responses, and pagination** can be used to acquire the data required by an AI-based heatwave prediction and early-warning system.
2. Irrigation Advisory System for Sugarcane Crop using AI and Sensor-based Technology	**Q2.** An AI-based sugarcane irrigation advisory system requires weather information such as rainfall, temperature, and humidity. Explain how an authenticated REST API can be used to retrieve these parameters and how the returned JSON data can subsequently be provided as input to an AI-based irrigation advisory model.
3. Urine Test Strip Reader using Colorimetry as a Point-of-Care Technology	**Q3.** A urine test strip analysis system stores colorimetric measurements and corresponding test results through a REST API. Explain how **API authentication, parameterized requests, JSON responses, and pagination** can be used to securely retrieve patient-test records for subsequent AI-based analysis.
4. AI-Based Autonomous Inter-Satellite Data Routing and Multi-Mode Amateur Radio Communication for PocketQube Missions	**Q4.** A PocketQube mission collects telemetry and communication data that are exposed through a REST API. Explain how **API documentation, authentication, query parameters, and pagination** can be used to retrieve the required telemetry data, which can then be supplied to an AI-based autonomous routing system

Program and Output with screenshots:

```
import requests

# ------------------------------------------------------------
# Experiment No. 5
# REST API Authentication, Query Parameters, JSON
# Response and Pagination
# ------------------------------------------------------------

# API endpoint
API_URL = "https://jsonplaceholder.typicode.com/posts"

# Store all retrieved records
all_posts = []

# Number of pages
total_pages = 3

# Records per page
page_size = 10

# ------------------------------------------------------------
# Fetch data page by page
# ------------------------------------------------------------

for page in range(1, total_pages + 1):

    # Query parameters
    params = {
        "_page": page,
        "_limit": page_size
    }

    # Send GET request
    response = requests.get(API_URL, params=params)

    print("\n----------------------------------------")
    print("Page:", page)
    print("Status Code:", response.status_code)

    # Check response
    if response.status_code == 200:

        # Convert JSON response into Python object
        data = response.json()

        # Add records to list
        all_posts.extend(data)
```

```
        print("Records Retrieved:", len(data))

        # Display records
        for post in data:

            print("\nPost ID:", post["id"])
            print("User ID:", post["userId"])
            print("Title:", post["title"])

    else:

        print("Error:", response.status_code)
        print(response.text)

# ------------------------------------------------------------
# Final output
# ------------------------------------------------------------

print("\n========================================")
print("TOTAL RECORDS COLLECTED:", len(all_posts))
print("========================================")
```

Output:

```
----------------------------------------
Page: 1
Status Code: 200
Records Retrieved: 10

Post ID: 1
User ID: 1
Title: sunt aut facere repellat provident occaecati excepturi optio reprehenderit

Post ID: 2
User ID: 1
Title: qui est esse

Post ID: 3
User ID: 1
Title: ea molestias quasi exercitationem repellat qui ipsa sit aut

...

Post ID: 10
User ID: 1
Title: optio molestias id quia eum
```

```
----------------------------------------
Page: 2
Status Code: 200
Records Retrieved: 10

Post ID: 11
User ID: 2
Title: et ea vero quia laudantium autem

Post ID: 12
User ID: 2
Title: in quibusdam tempore odit est dolorem

...

Post ID: 20
User ID: 2
Title: ullam nobis libero sapiente ad optio sint


----------------------------------------
Page: 3
Status Code: 200
Records Retrieved: 10

Post ID: 21
User ID: 3
Title: rerum debitis voluptatem qui eveniet tempora distinctio

...

Post ID: 30
User ID: 3
Title: natus quaerat id

========================================
TOTAL RECORDS COLLECTED: 30
========================================
```

Result

Thus, REST APIs were authenticated and consumed using an API key, query parameters and pagination. The JSON responses were retrieved, processed, and relevant information was extracted successfully.